

Mini RFC: Reliable Transmission

April 5, 2022

Note: This is a simplified and annotated summary of the QUIC-related IETF RFCs. It's part of a course project at Tsinghua University and not suitable for any other purposes.

We reorder paragraphs in QUIC RFCs and omit many details to improve its readability for beginners.

WARNING: DO NOT distribute this document. Modification of extracts from IETF RFCs is not granted by IETF as original authors retain copyrights of RFCs. This document is for educational purposes only.

1 Overview

All transmissions in QUIC are sent with a packet-level header, which indicates the encryption level and includes a packet sequence number (referred to below as a packet number). The encryption level indicates the packet number space. Packet numbers never repeat within a packet number space for the lifetime of a connection. Packet numbers are sent in monotonically increasing order within a space, preventing ambiguity. It is permitted for some packet numbers to never be used, leaving intentional gaps.

This design obviates the need for disambiguating between transmissions and retransmissions; this eliminates significant complexity from QUIC's interpretation of TCP loss detection mechanisms.

QUIC packets can contain multiple frames of different types. The recovery mechanisms ensure that data and frames that need reliable delivery are acknowledged or declared lost and sent in new packets as necessary.

We only use one packet number space since we use only one encryption level: no encryption.

Allowing gaps in packet numbers encourages more robust implementations and may also be explicitly used to control the behavior of QUIC in some scenarios.

2 Packet Number

The packet number is an integer in the range 0 to $2^{62} - 1$. Each endpoint maintains a separate packet number for sending and receiving.

Packet numbers start at packet number 0 . Subsequent packets sent MUST increase the packet number by at least one.

A QUIC endpoint MUST NOT reuse a packet number within the same packet number space in one connection. If the packet number for sending reaches $2^{62} - 1$, the sender MUST close the connection.

TCP conflates transmission order at the sender with delivery order at the receiver, resulting in the retransmission ambiguity problem. QUIC separates transmission order from delivery order: packet numbers indicate transmission order, and delivery order is determined by the stream offsets in STREAM frames.

QUIC's packet number is strictly increasing within a packet number space and directly encodes transmission order. A higher packet number signifies that the packet was sent later, and a lower packet number signifies that the packet was sent earlier. When a packet is detected lost, QUIC includes necessary frames in a new packet with a new packet number, removing ambiguity about which packet is acknowledged when an ACK is received. Consequently, more accurate RTT measurements can be made, spurious retransmissions are trivially detected, and mechanisms such as Fast Retransmit can be applied universally, based only on packet number.

This design point significantly simplifies loss detection mechanisms for QUIC. Most TCP mechanisms implicitly attempt to infer transmission ordering based on TCP sequence numbers – a nontrivial task, especially when TCP timestamps are not available.

QUIC acknowledges packets, while TCP acknowledges data in the ordered byte stream.

3 Acknowledgments

3.1 Ack-eliciting and Non-ack-eliciting

All frames other than ACK, PADDING, and CONNECTION_CLOSE are considered ack-eliciting. Packets that contain ack-eliciting frames are called ack-eliciting packets.

Endpoints acknowledge **all** packets they receive and process. However, only ack-eliciting packets cause an ACK frame to be sent within the maximum ack delay. Packets that are not ack-eliciting are only acknowledged when an ACK frame is sent for other reasons.

Every packet SHOULD be acknowledged at least once, and ack-eliciting packets MUST be acknowledged at least once within the maximum delay an endpoint communicated **using the `max_ack_delay` transport parameter**. `max_ack_delay` declares an explicit contract: an endpoint promises to never **intentionally** delay acknowledgments of an ack-eliciting packet by more than the indicated value. If it does, any excess accrues to the RTT estimate and could result in spurious or delayed retransmissions from the peer. A sender uses the receiver's `max_ack_delay` value in determining timeouts for timer-based retransmission.

The default of 25 milliseconds is assumed.

On a non-real-time OS, like Linux, it's impossible for a QUIC implementation to guarantee that a packet is acknowledged in time. It is a "best-effort" service.

An endpoint MUST NOT send a non-ack-eliciting packet in response to a non-ack-eliciting packet, even if there are packet gaps that precede the received packet. This avoids an infinite feedback loop of acknowledgments, which could prevent the connection from ever becoming idle. Non-ack-eliciting packets are eventually acknowledged when the endpoint sends an ACK frame in response to other events. When only non-ack-eliciting packets need to be acknowledged, an endpoint MAY choose not to send an ACK

frame with outgoing frames until an ack-eliciting packet has been received.

An endpoint that is only sending ACK frames will not receive acknowledgments from its peer unless those acknowledgments are included in packets with ack-eliciting frames. An endpoint that is only sending non-ack-eliciting packets might choose to occasionally add an ack-eliciting frame to those packets to ensure that it receives an acknowledgment.

Such as the PING frame

3.2 ACK Frequency

In general, frequent feedback from a receiver improves loss and congestion response, but this has to be balanced against excessive load generated by a receiver that sends an ACK frame in response to every ack-eliciting packet. The guidance offered below seeks to strike this balance. A receiver **SHOULD** send an ACK frame after receiving at least two ack-eliciting packets. This recommendation is general in nature and consistent with recommendations for TCP endpoint behavior.

Deciding the acknowledgment frequency is a touch problem deserving a separate RFC.

In order to assist loss detection at the sender, an endpoint **SHOULD** generate and send an ACK frame without delay when it receives an ack-eliciting packet either:

- when the received packet has a packet number less than another ack-eliciting packet that has been received, or
- when the packet has a packet number larger than the highest-numbered ack-eliciting packet that has been received and there are missing packets between that packet and this packet.

When sending a packet for any reason, an endpoint **SHOULD** attempt to include an ACK frame if one has not been sent recently. Doing so helps with timely loss detection at the peer.

3.3 ACK Ranges

When an ACK frame is sent, one or more ranges of acknowledged packets are included. Including acknowledgments for older packets reduces the chance of spurious retransmissions caused by losing previously sent ACK frames, at the cost of larger ACK frames.

ACK frames **SHOULD** always acknowledge the most recently received packets, and the more out of order the packets are, the more important it is to send an updated ACK frame quickly, to prevent the peer from declaring a packet as lost and spuriously retransmitting the frames it contains. An ACK frame is expected to fit within a single QUIC packet. If it does not, then older ranges (those with the smallest packet numbers) are omitted.

A receiver limits the number of ACK Ranges it remembers and sends in ACK frames, both to limit the size of ACK frames and to avoid resource exhaustion. After receiving acknowledgments for an ACK frame, the receiver

Check §7.3.2 to learn the format of ACK Frames.

SHOULD stop tracking those acknowledged ACK Ranges. Senders can expect acknowledgments for most packets, but QUIC does not guarantee receipt of an acknowledgment for every packet that the receiver processes.

Peer receives (packets containing) ACK (frames) of (sending packets that contain) ACK (frames).

When a packet containing an ACK frame is sent, the Largest Acknowledged field in that frame can be saved. When a packet containing an ACK frame is acknowledged, the receiver can stop acknowledging packets less than or equal to the Largest Acknowledged field in the sent ACK frame.

Acknowledgments may be lost and lead to spurious retransmissions.

It is possible that retaining many ACK Ranges could cause an ACK frame to become too large. A receiver can discard unacknowledged ACK Ranges to limit ACK frame size, at the cost of increased retransmissions from the sender. This is necessary if an ACK frame would be too large to fit in a packet. Receivers MAY also limit ACK frame size further to preserve space for other frames or to limit the capacity that acknowledgments consume.

A receiver MUST retain an ACK Range unless it can ensure that it will not subsequently accept packets with numbers in that range. Maintaining a minimum packet number that increases as ranges are discarded is one way to achieve this with minimal state.

Receivers can discard all ACK Ranges, but they MUST retain the largest packet number that has been successfully processed, as that is used to recover packet numbers from subsequent packets.

3.4 Measuring and Reporting Host Delay

An endpoint measures the delays intentionally introduced between the time the packet with the largest packet number is received and the time an acknowledgment is sent. The endpoint encodes this acknowledgment delay in the ACK Delay field of an ACK frame. This allows the receiver of the ACK frame to adjust for any intentional delays, which is important for getting a better estimate of the path RTT when acknowledgments are delayed.

A packet might be held in the OS kernel or elsewhere on the host before being processed. An endpoint MUST NOT include delays that it does not control when populating the ACK Delay field in an ACK frame. When the measured acknowledgment delay is larger than its `max_ack_delay`, an endpoint SHOULD report the measured delay.

In practice, however, it is difficult for a process to get the time it is not scheduled.

4 Estimating the Round-Trip Time

At a high level, an endpoint measures the time from when a packet was sent to when it is acknowledged as an RTT sample. The endpoint uses RTT samples and peer-reported host delays to generate a statistical description of the network path's RTT. An endpoint computes the following three values for each path: the minimum value over a period of time (`min_rtt`), an exponentially weighted moving average (`smoothed_rtt`), and the mean deviation (referred to as "variation" in the rest of this document) in the observed RTT samples (`rttvar`).

4.1 Generating RTT Samples

An endpoint generates an RTT sample on receiving an ACK frame that meets the following two conditions:

- the largest acknowledged packet number is newly acknowledged
- at least one of the newly acknowledged packets was ack-eliciting

The RTT sample, `latest_rtt`, is generated as the time elapsed since the largest acknowledged packet was sent:

$$\text{latest_rtt} = \text{ack_time} - \text{send_time_of_largest_acked}$$

An RTT sample is generated using only the largest acknowledged packet in the received ACK frame. This is because a peer reports acknowledgment delays for only the largest acknowledged packet in an ACK frame. While the reported acknowledgment delay is not used by the RTT sample measurement, it is used to adjust the RTT sample in subsequent computations of `smoothed_rtt` and `rttvar`.

To avoid generating multiple RTT samples for a single packet, an ACK frame **SHOULD NOT** be used to update RTT estimates if it does not newly acknowledge the largest acknowledged packet.

An RTT sample **MUST NOT** be generated on receiving an ACK frame that does not newly acknowledge at least one ack-eliciting packet. A peer usually does not send an ACK frame when only non-ack-eliciting packets are received. Therefore, an ACK frame that contains acknowledgments for only non-ack-eliciting packets could include an arbitrarily large ACK Delay value. Ignoring such ACK frames avoids complications in subsequent `smoothed_rtt` and `rttvar` computations.

4.2 Estimating `min_rtt`

`min_rtt` is the sender's estimate of the minimum RTT observed for a given network path over a period of time. In this document, `min_rtt` is used by loss detection to reject implausibly small RTT samples.

`min_rtt` **MUST** be set to the `latest_rtt` on the first RTT sample. `min_rtt` **MUST** be set to the lesser of `min_rtt` and `latest_rtt` on all other samples.

An endpoint uses only locally observed times in computing the `min_rtt` and does not adjust for acknowledgment delays reported by the peer. Doing so allows the endpoint to set a lower bound for the `smoothed_rtt` based entirely on what it observes and limits potential underestimation due to erroneously reported delays by the peer.

The RTT for a network path may change over time. If a path's actual RTT decreases, the `min_rtt` will adapt immediately on the first low sample. If the path's actual RTT increases, however, the `min_rtt` will not adapt to it, allowing future RTT samples that are smaller than the new RTT to be included in `smoothed_rtt`.

4.3 Estimating smoothed_rtt and var

smoothed_rtt is an exponentially weighted moving average of an endpoint's RTT samples, and rttvar estimates the variation in the RTT samples using a mean variation.

The calculation of smoothed_rtt uses RTT samples after adjusting them for acknowledgment delays. These delays are decoded from the ACK Delay field of ACK frames.

Any acknowledgment delays reported by the peer that are greater than the peer's max_ack_delay are attributed to unintentional but potentially repeating delays, such as scheduler latency at the peer or loss of previous acknowledgments. Excess delays could also be due to a noncompliant receiver. Therefore, these extra delays are considered effectively part of path delay and incorporated into the RTT estimate.

Therefore, when adjusting an RTT sample using peer-reported acknowledgment delays, an endpoint MUST NOT subtract the acknowledgment delay from the RTT sample if the resulting value is smaller than the min_rtt. This limits the underestimation of the smoothed_rtt due to a misreporting peer.

Similar to [RFC6298] (Computing TCP's Retransmission Timer), smoothed_rtt and rttvar are computed as follows.

An endpoint initializes the RTT estimator during connection establishment. Before any RTT samples are available for a new path, the estimator is initialized using the initial RTT.

smoothed_rtt and rttvar are initialized as follows, where kInitialRtt contains the initial RTT value:

```
smoothed_rtt = kInitialRtt
rttvar = kInitialRtt / 2
```

The initial RTT (kInitialRtt) SHOULD be set to 333 milliseconds.

It's a value specified by the RFC.

RTT samples for the network path are recorded in latest_rtt. On the first RTT sample after initialization, the estimator is reset using that sample. On the first RTT sample after initialization, smoothed_rtt and rttvar are set as follows:

```
smoothed_rtt = latest_rtt
rttvar = latest_rtt / 2
```

On subsequent RTT samples, smoothed_rtt and rttvar evolve as follows:

Note that one endpoint does not trust that its peer will report a correct ACK delay.

```
ack_delay = min(ack_delay, max_ack_delay)
adjusted_rtt = latest_rtt
if (latest_rtt ≥ min_rtt + ack_delay):
    adjusted_rtt = latest_rtt - ack_delay
smoothed_rtt = 7/8 * smoothed_rtt + 1/8 * adjusted_rtt
```

```
rttvar_sample = abs(smoothed_rtt - adjusted_rtt)
rttvar = 3/4 * rttvar + 1/4 * rttvar_sample
```

5 Loss Detection

QUIC senders use acknowledgments to detect lost packets. If a packet is lost, the QUIC transport needs to recover from that loss, such as by retransmitting the data, sending an updated frame, or discarding the frame.

Acknowledgment-based loss detection implements the spirit of TCP's Fast Retransmit [RFC5681], Early Retransmit [RFC5827], Forward Acknowledgment [FACK], SACK loss recovery [RFC6675], and RACK-TLP [RFC8985]. This section provides an overview of how these algorithms are implemented in QUIC.

QUIC learns many things from the success of TCP.

A packet is declared lost if it meets all of the following conditions:

- The packet is unacknowledged, in flight, and was sent prior to an acknowledged packet.
- The packet was sent `kPacketThreshold` packets before an acknowledged packet, or it was sent long enough in the past.

The acknowledgment indicates that a packet sent later was delivered, and the packet and time thresholds provide some tolerance for packet reordering.

Spuriously declaring packets as lost leads to unnecessary retransmissions and may result in degraded performance due to the actions of the congestion controller upon detecting loss. Implementations can detect spurious retransmissions and increase the packet or time reordering threshold to reduce future spurious retransmissions and loss events. Implementations with adaptive time thresholds MAY choose to start with smaller initial reordering thresholds to minimize recovery latency.

5.1 Packet Threshold

The RECOMMENDED initial value for the packet reordering threshold (`kPacketThreshold`) is 3, based on best practices for TCP loss detection [RFC5681] [RFC6675]. In order to remain similar to TCP, implementations SHOULD NOT use a packet threshold less than 3.

Some networks may exhibit higher degrees of packet reordering, causing a sender to detect spurious losses. Additionally, packet reordering could be more common with QUIC than TCP because network elements that could observe and reorder TCP packets cannot do that for QUIC and also because QUIC packet numbers are encrypted. Algorithms that increase the reordering threshold after spuriously detecting losses, such as RACK [RFC8985], have proven to be useful in TCP and are expected to be at least as useful in QUIC.

5.2 Time Threshold

Once a later packet has been acknowledged, an endpoint SHOULD declare an earlier packet lost if it was sent a threshold amount of time in the past. To avoid declaring packets as lost too early, this time threshold MUST be set to at least the local timer granularity, as indicated by the `kGranularity` constant. The time threshold is:

$$\max(kTimeThreshold * \max(smoothed_rtt, latest_rtt), kGranularity)$$

If packets sent prior to the largest acknowledged packet cannot yet be declared lost, then a timer SHOULD be set for the remaining time.

Using $\max(smoothed_rtt, latest_rtt)$ protects from the two following cases:

- the latest RTT sample is lower than the smoothed RTT, perhaps due to reordering where the acknowledgment encountered a shorter path;
- the latest RTT sample is higher than the smoothed RTT, perhaps due to a sustained increase in the actual RTT, but the smoothed RTT has not yet caught up.

The RECOMMENDED time threshold (`kTimeThreshold`), expressed as an RTT multiplier, is 9/8. The RECOMMENDED value of the timer granularity (`kGranularity`) is 1 millisecond.

6 Retransmissions

QUIC packets that are determined to be lost are not retransmitted whole. The same applies to the frames that are contained within lost packets. Instead, the information that might be carried in frames is sent again in new frames as needed.

New frames and packets are used to carry information that is determined to have been lost. In general, information is sent again when a packet containing that information is determined to be lost, and sending ceases when a packet containing that information is acknowledged.

- Application data sent in STREAM frames is retransmitted in new STREAM frames.
- ACK frames carry the most recent set of acknowledgments and the acknowledgment delay from the largest acknowledged packet. Delaying the transmission of packets containing ACK frames or resending old ACK frames can cause the peer to generate an inflated RTT sample or unnecessarily disable ECN.
- PING and PADDING frames contain no information, so lost PING or PADDING frames do not require repair.

7 Packets and Frames

7.1 Packet Number Encoding

When present in long or short packet headers, packet numbers are encoded in 1 to 4 bytes. The number of bits required to represent the packet number is reduced by including only the least significant bits of the packet number.

Prior to receiving an acknowledgment, the full packet number **MUST** be included; it is not to be truncated, as described below.

After an acknowledgment is received, the sender **MUST** use a packet number size able to represent more than twice as large a range as the difference between the largest acknowledged packet number and the packet number being sent. A peer receiving the packet will then correctly decode the packet number, unless the packet is delayed in transit such that it arrives after many higher-numbered packets have been received. An endpoint **SHOULD** use a large enough packet number encoding to allow the packet number to be recovered even if the packet arrives after packets that are sent afterwards.

As a result, the size of the packet number encoding is at least one bit more than the base-2 logarithm of the number of contiguous unacknowledged packet numbers, including the new packet.

At a receiver, The full packet number is then reconstructed based on the number of significant bits present, the value of those bits, and the largest packet number received.

You can skip this subsection at your first read and read it later when you set about writing code.

7.1.1 encoding pseudocode

The pseudocode in Figure 1 shows how an implementation can select an appropriate size for packet number encodings.

The `EncodePacketNumber` function takes two arguments:

- `full_pn` is the full packet number of the packet being sent.
- `largest_acked` is the largest packet number that has been acknowledged by the peer in the current packet number space, if any.

7.1.2 decoding pseudocode

The pseudocode in Figure 2 includes an example algorithm for decoding packet numbers.

The `DecodePacketNumber` function takes three arguments:

- `largest_pn` is the largest packet number that has been successfully processed in the current packet number space.
- `truncated_pn` is the value of the Packet Number field.
- `pn_nbits` is the number of bits in the Packet Number field (8, 16, 24, or 32).

```

def EncodePacketNumber(full_pn, largest_acked):
    # The number of bits must be at least one more
    # than the base-2 logarithm of the number of contiguous
    # unacknowledged packet numbers, including the new packet.
    if largest_acked is None:
        num_unacked = full_pn + 1
    else:
        num_unacked = full_pn - largest_acked

    min_bits = log(num_unacked, 2) + 1
    num_bytes = ceil(min_bits / 8)

    # Encode the integer value and truncate to
    # the num_bytes least significant bytes.
    return encode(full_pn, num_bytes)

```

Figure 1: *Sample Packet Number Encoding Algorithm*

7.2 Packets

There is no new type of packets introduced in this RFC.

7.3 Frames

7.3.1 PING Frames

Endpoints can use PING frames (type=0x01) to verify that their peers are still alive or to check reachability to the peer.

PING frames are formatted as shown in Figure 3, which shows that PING frames have no content.

<pre> PING Frame { Type (i) = 0x01, } </pre>
--

Figure 3: *PING Frame Format*

The receiver of a PING frame simply needs to acknowledge the packet containing this frame.

The PING frame can be used to keep a connection alive when an application or application protocol wishes to prevent the connection from timing out.

```

def DecodePacketNumber(largest_pn, truncated_pn, pn_nbits):
    expected_pn = largest_pn + 1
    pn_win      = 1 << pn_nbits
    pn_hwin     = pn_win / 2
    pn_mask     = pn_win - 1
    # The incoming packet number should be greater than
    # expected_pn - pn_hwin and less than or equal to
    # expected_pn + pn_hwin
    #
    # This means we cannot just strip the trailing bits from
    # expected_pn and add the truncated_pn because that might
    # yield a value outside the window.
    #
    # The following code calculates a candidate value and
    # makes sure it's within the packet number window.
    # Note the extra checks to prevent overflow and underflow.
    candidate_pn = (expected_pn & ~pn_mask) | truncated_pn
    if candidate_pn ≤ expected_pn - pn_hwin and
        candidate_pn < (1 << 62) - pn_win:
        return candidate_pn + pn_win
    if candidate_pn > expected_pn + pn_hwin and
        candidate_pn ≥ pn_win:
        return candidate_pn - pn_win
    return candidate_pn

```

Figure 2: Sample Packet Number Decoding Algorithm

7.3.2 ACK Frames

Receivers send ACK frames (types 0x02) to inform senders of packets they have received and processed. The ACK frame contains one or more ACK Ranges. ACK Ranges identify acknowledged packets.

QUIC acknowledgments are irrevocable. Once acknowledged, a packet remains acknowledged, even if it does not appear in a future ACK frame. This is unlike reneging for TCP Selective Acknowledgments (SACKs).

Packets from different packet number spaces can be identified using the same numeric value. An acknowledgment for a packet needs to indicate both a packet number and a packet number space. This is accomplished by having each ACK frame only acknowledge packet numbers in the same space as the packet in which the ACK frame is contained.

ACK frames are formatted as shown in Figure 4.

Again, we use only one packet number space.

- **Largest Acknowledged:** A variable-length integer representing the largest packet number the peer is acknowledging; this is usually the largest packet number that the peer has received prior to generating the ACK frame. Un-

```

ACK Frame {
  Type (i) = 0x02,
  Largest Acknowledged (i),
  ACK Delay (i),
  ACK Range Count (i),
  First ACK Range (i),
  ACK Range (..) ...,
}

```

Figure 4: *ACK Frame Format*

like the packet number in the QUIC long or short header, the value in an ACK frame is not truncated.

- **ACK Delay:** A variable-length integer encoding the acknowledgment delay in microseconds. It is decoded by multiplying the value in the field by 2 to the power of the `ack_delay_exponent` transport parameter sent by the sender of the ACK frame. Compared to simply expressing the delay as an integer, this encoding allows for a larger range of values within the same number of bytes, at the cost of lower resolution.
- **ACK Range Count:** A variable-length integer specifying the number of ACK Range fields in the frame.
- **First ACK Range:** A variable-length integer indicating the number of contiguous packets preceding the Largest Acknowledged that are being acknowledged. That is, the smallest packet acknowledged in the range is determined by subtracting the First ACK Range value from the Largest Acknowledged field.
- **ACK Ranges:** Contains additional ranges of packets that are alternately not acknowledged (Gap) and acknowledged (ACK Range).

The default value
`ack_delay_exponent = 3` is applied.

Each ACK Range consists of alternating Gap and ACK Range Length values in descending packet number order. ACK Ranges can be repeated. The number of Gap and ACK Range Length values is determined by the ACK Range Count field; one of each value is present for each value in the ACK Range Count field.

ACK Ranges are structured as shown in Figure 5.

The fields that form each ACK Range are:

- **Gap:** A variable-length integer indicating the number of contiguous unacknowledged packets preceding the packet number one lower than the smallest in the preceding ACK Range.
- **ACK Range Length:** A variable-length integer indicating the number of contiguous acknowledged packets preceding the largest packet number, as determined by the preceding Gap.

```

ACK Range {
    Gap (i),
    ACK Range Length (i),
}

```

Figure 5: *ACK Ranges*

Gap and ACK Range Length values use a relative integer encoding for efficiency. Though each encoded value is positive, the values are subtracted, so that each ACK Range describes progressively lower-numbered packets.

Each ACK Range acknowledges a contiguous range of packets by indicating the number of acknowledged packets that precede the largest packet number in that range. A value of 0 indicates that only the largest packet number is acknowledged. Larger ACK Range values indicate a larger range, with corresponding lower values for the smallest packet number in the range. Thus, given a largest packet number for the range, the smallest value is determined by the following formula:

$$\text{smallest} = \text{largest} - \text{ack_range}$$

An ACK Range acknowledges all packets between the smallest packet number and the largest, inclusive.

The largest value for an ACK Range is determined by cumulatively subtracting the size of all preceding ACK Range Lengths and Gaps.

Each Gap indicates a range of packets that are not being acknowledged. The number of packets in the gap is one higher than the encoded value of the Gap field.

The value of the Gap field establishes the largest packet number value for the subsequent ACK Range using the following formula:

$$\text{largest} = \text{previous_smallest} - \text{gap} - 2$$

For example, if we want to acknowledge [0, 3], [5, 7], [10, 12], we need to send a ACK frame as below:

```

ACK Frame {
    Type (i) = 0x02,
    Largest Acknowledged (i) = 12,
    ACK Delay (i) = 25 ms,
    ACK Range Count (i) = 2, // [0, 3], [5, 7]
    First ACK Range (i) = 2, // 12 - 2 = 10
    ACK Range = [
        ACK Range {
            Gap (i) = 1 // 10 - 1 - 2 = 7
            ACK Range Length (i) = 2 // 7 - 2 = 5
        },
    ],
}

```

```
ACK Range {  
    Gap (i) = 0 // 5 - 0 - 2 = 3  
    ACK Range Length (i) = 3 // 3 - 3 = 0  
}  
],  
}
```

8 Application Protocol

[There is no new application protocol defined in this mini RFC.](#)